Labsheet 4

1.Write a C++ program to create and display a Singly Linked List.

```
int main31() {
    LNode* head = new LNode(10);
    head->next = new LNode(20);
    head->next->next = new LNode(30);

    displayList(head);

    return 0;
}
```

2. Write a C++ program to insert a node at the beginning of a Linked List.

```
int main32() {
    LNode* head = new LNode(20);
    head->next = new LNode(30);

    LNode* newNode = new LNode(10);
    newNode->next = head;
    head = newNode;

    displayList(head);

    return 0;
}
```

3. Write a C++ program to insert a node at the end of a Linked List.

```
int main33() {
    LNode* head = new LNode(10);
    head->next = new LNode(20);

    LNode* newNode = new LNode(30);
    LNode* temp = head;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;

    displayList(head);

    return 0;
}
```

4. Write a C++ program to insert a node at a given position.

```
int main34() {
    LNode* head = new LNode(10);
    head->next = new LNode(20);
    head->next->next = new LNode(30);

    int value, position;
    cout << "Enter value: ";
    cin >> value;
    cout << "Enter position: ";
    cin >> position;

    if (position == 1) {
        LNode* newNode = new LNode(value);
        newNode->next = head;
        head = newNode;
    }
    else {
        LNode* temp = head;

        for (int i = 1; i < position - 1 && temp != NULL; i++)
            temp = temp->next;

        if (temp == NULL) {
            cout << "Invalid position";
            return 0;
        }

        LNode* newNode = new LNode(value);
        newNode->next = temp->next;
        temp->next = newNode;
    }
```

```
    displayList(head);

    return 0;
}
```

5. Write a C++ program to delete a node from the beginning.

```
int main35() {
    LNode* head = new LNode(10);
    head->next = new LNode(20);

    if (head == NULL) {
        cout << "List is Empty";
        return 0;
    }

    LNode* temp = head;
    head = head->next;
    delete temp;

    displayList(head);

    return 0;
}
```

6. Write a C++ program to delete a node from the end.

```
int main36() {
    LNode* head = new LNode(10);
    head->next = new LNode(20);
    head->next->next = new LNode(30);

    if (head == NULL) {
        cout << "List is Empty";
        return 0;
    }

    if (head->next == NULL) {
        delete head;
        head = NULL;
    }
    else {
        LNode* temp = head;

        while (temp->next->next != NULL)
            temp = temp->next;

        delete temp->next;
        temp->next = NULL;
    }

    displayList(head);

    return 0;
}
```

7. Write a C++ program to delete a node from a given position.

```
int main37() {
    LNode* head = new LNode(10);
    head->next = new LNode(20);
    head->next->next = new LNode(30);

    int position;
    cout << "Enter position to delete: ";
    cin >> position;

    if (head == NULL) {
        cout << "List is Empty";
        return 0;
    }

    if (position == 1) {
        LNode* temp = head;
        head = head->next;
        delete temp;
    }
    else {
        LNode* temp = head;

        for (int i = 1; i < position - 1 && temp != NULL; i++)
            temp = temp->next;

        if (temp == NULL || temp->next == NULL) {
            cout << "Invalid position";
            return 0;
        }
```

```
        LNode* deleteNode = temp->next;
        temp->next = deleteNode->next;
        delete deleteNode;
    }

    displayList(head);

    return 0;
}
```

8. Write a C++ program to search for an element in a Singly Linked List.

```
int main38() {
    LNode* head = new LNode(10);
    head->next = new LNode(20);
    head->next->next = new LNode(30);

    int key;
    cout << "Enter key: ";
    cin >> key;

    LNode* temp = head;
    int position = 1;

    while (temp != NULL) {
        if (temp->data == key) {
            cout << "Found at position " << position;
            return 0;
        }

        temp = temp->next;
        position++;
    }

    cout << "Not found";

    return 0;
}
```

9. Write a C++ program to count the total number of nodes in a Linked List.

```
int main39() {
    LNode* head = new LNode(10);
    head->next = new LNode(20);
    head->next->next = new LNode(30);

    int count = 0;
    LNode* temp = head;

    while (temp != NULL) {
        count++;
        temp = temp->next;
    }

    cout << "Total nodes = " << count;

    return 0;
}
```

10. Write a C++ program to find the maximum and minimum element in a Linked List.

```
int main40() {
    LNode* head = new LNode(10);
    head->next = new LNode(30);
    head->next->next = new LNode(20);

    int maximum = head->data;
    int minimum = head->data;

    LNode* temp = head->next;

    while (temp != NULL) {
        if (temp->data > maximum)
            maximum = temp->data;

        if (temp->data < minimum)
            minimum = temp->data;

        temp = temp->next;
    }

    cout << "Maximum = " << maximum << endl
    cout << "Minimum = " << minimum << endl

    return 0;
}
```

11. Write a C++ program to reverse a Singly Linked List.

```
int main41() {
    LNode* head = new LNode(10);
    head->next = new LNode(20);
    head->next->next = new LNode(30);

    LNode* previous = NULL;
    LNode* current = head;

    while (current != NULL) {
        LNode* nextNode = current->next;
        current->next = previous;
        previous = current;
        current = nextNode;
    }

    head = previous;

    displayList(head);

    return 0;
}
```

12. Write a C++ program to sort the elements of a Linked List in ascending order.

```cpp
int main42() {
    LNode* head = new LNode(30);
    head->next = new LNode(10);
    head->next->next = new LNode(20);

    for (LNode* i = head; i != NULL; i = i->next) {
        for (LNode* j = i->next; j != NULL; j = j->next) {
            if (i->data > j->data)
                swap(i->data, j->data);
        }
    }

    displayList(head);

    return 0;
}
```

13. Write a menu-driven C++ program implementing insertion, deletion, searching and display operations.

```cpp
int main43() {
    LNode* head = NULL;
    int choice, value;

    do {
        cout << "\n1.Insert Beginning\n2.Insert End\n3.Delete Beginning\n";
        cout << "4.Delete End\n5.Search\n6.Display\n7.Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        if (choice == 1) {
            cout << "Enter value: ";
            cin >> value;

            LNode* newNode = new LNode(value);
            newNode->next = head;
            head = newNode;
        }
        else if (choice == 2) {
            cout << "Enter value: ";
            cin >> value;

            LNode* newNode = new LNode(value);

            if (head == NULL)
                head = newNode;
            else {
                LNode* temp = head;

                while (temp->next != NULL)
                    temp = temp->next;

                temp->next = newNode;
            }
        }
```

```
else if (choice == 3) {
    if (head == NULL)
        cout << "List is Empty";
    else {
        LNode* temp = head;
        head = head->next;
        delete temp;
    }
}
else if (choice == 4) {
    if (head == NULL)
        cout << "List is Empty";
    else if (head->next == NULL) {
        delete head;
        head = NULL;
    }
    else {
        LNode* temp = head;

        while (temp->next->next != NULL)
            temp = temp->next;

        delete temp->next;
        temp->next = NULL;
    }
}
else if (choice == 5) {
    cout << "Enter value: ";
    cin >> value;
```

```
            LNode* temp = head;
            int position = 1;
            bool found = false;

            while (temp != NULL) {
                if (temp->data == value) {
                    cout << "Found at position " << position;
                    found = true;
                    break;
                }

                temp = temp->next;
                position++;
            }

            if (!found)
                cout << "Not found";
        }
        else if (choice == 6) {
            displayList(head);
        }
        else if (choice != 7) {
            cout << "Invalid choice";
        }

    } while (choice != 7);

    return 0;
}
```

14. Write a C++ program to implement a Doubly Linked List and perform insertion and deletion operations.

```cpp
struct DNode {
    int data;
    DNode* prev;
    DNode* next;

    DNode(int value) {
        data = value;
        prev = NULL;
        next = NULL;
    }
};

int main44() {
    DNode* first = new DNode(10);
    DNode* second = new DNode(20);

    first->next = second;
    second->prev = first;

    cout << "Doubly Linked List: ";
    cout << first->data << " <-> " << second->data << endl;

    // Delete second node
    first->next = NULL;
    delete second;

    cout << "After deletion: " << first->data << endl;

    return 0;
}
```

15. Write a C++ program to implement a Circular Linked List and perform traversal and insertion operations.

```cpp
int main45() {
    LNode* first = new LNode(10);
    LNode* second = new LNode(20);
    LNode* third = new LNode(30);

    first->next = second;
    second->next = third;
    third->next = first;

    LNode* newNode = new LNode(40);
    newNode->next = first;
    third->next = newNode;

    cout << "Circular List: ";

    LNode* temp = first;

    do {
        cout << temp->data << " ";
        temp = temp->next;
    } while (temp != first);

    return 0;
}
```

+